

Advanced Agentic AI Systems Engineering

5-Day Advanced Training Program

هندسة أنظمة الذكاء الاصطناعي الوكيل المتقدمة

June 2026



SDAIA

الهيئة السعودية للبيانات
والذكاء الاصطناعي
Saudi Data & AI Authority



Course Introduction

► Overall Goal

Enable **advanced engineers** to **design, secure, optimize, monitor, and deploy** enterprise-grade **Agentic AI systems** while balancing operational autonomy with governance, security, and scalability.



Target Audience

- AI Engineers
- Software Architects
- Agentic Systems Developers
- Government Digital Transformation Teams
- Cloud Platform Engineers.





Learning Outcomes

- **Design** complex agentic systems using advanced architectural patterns.
- **Develop** integrated agentic workflows with effective state management and data flow.
- **Build** automated guardrails and test system resistance to attacks.
- **Optimize** performance and cost of agentic systems and monitor them in production using tracing and intelligent recall tools.
- **Apply** enterprise challenge solving methodologies by building and deploying a full agentic system in a simulated environment.





PRE COURSE REQUIREMENTS

Participants should already know

- Python fundamentals
- APIs and REST
- Basic LLM concepts
- Prompt engineering basics
- Cloud fundamentals
- Git and GitHub basics
- Software architecture basics



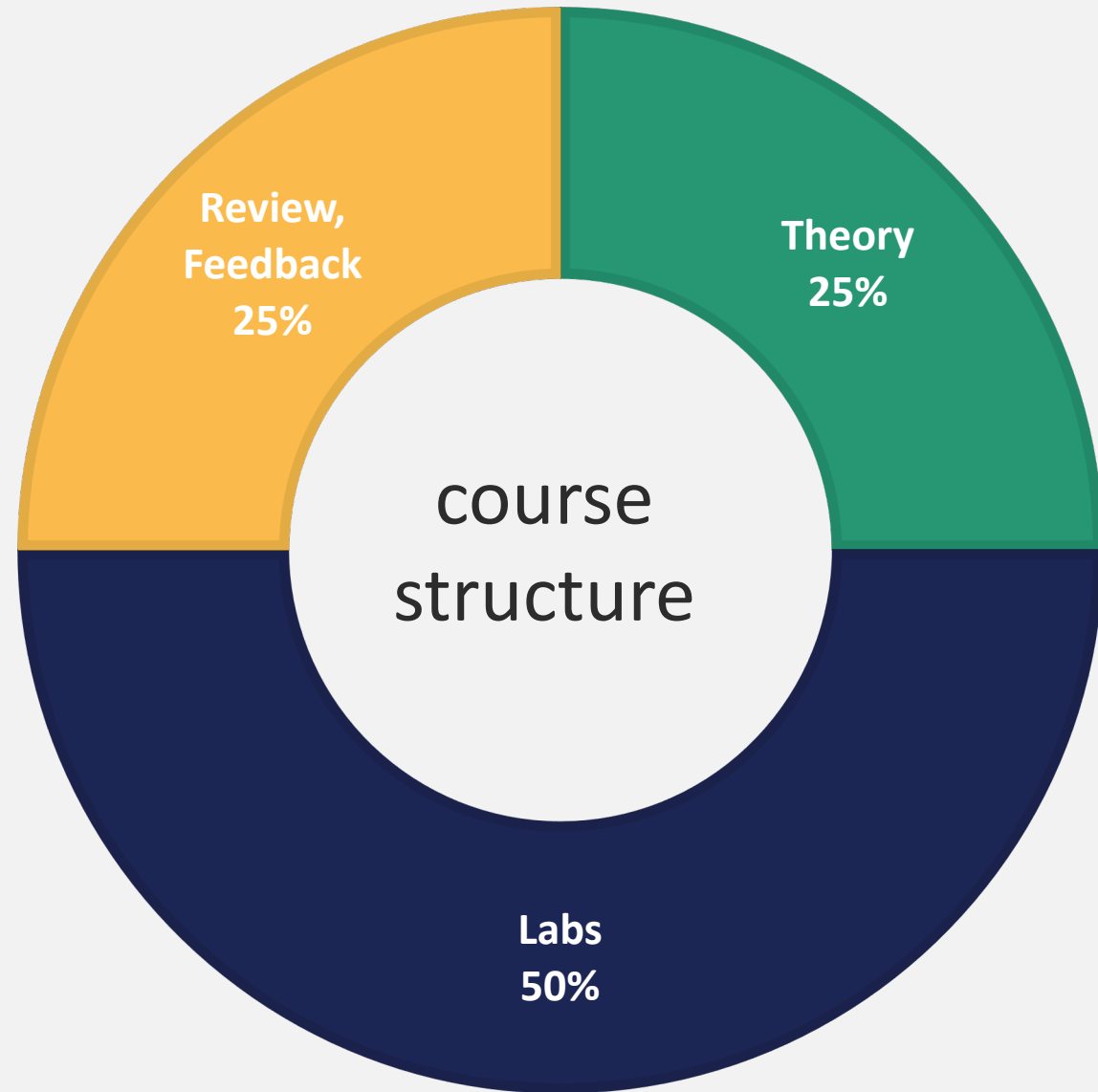
Course daily structure

Duration

5 Days

6 Hours a day

30 Total Training Hours



Practical Deliverables

- **Deployable multi-agent source code**
- **Security and pen testing reports**
- **Monitoring and cost optimization dashboards**
- **Final capstone project**



▶ Recommended Hardware Requirements

Device Type	Minimum Recommended Specs	Preferred Specs
Windows PC	Intel i5 / Ryzen 5, 16 GB RAM, 50 GB SSD Free Space, Windows 10/11	Intel i7 / Ryzen 7, 32 GB RAM, 100+ GB SSD
MacBook / macOS	Apple M1 or Intel i5, 16 GB RAM, 50 GB SSD Free Space	Apple M2/M3 Pro, 24+ GB RAM, 100+ GB SSD
General Requirements	Stable Internet, Python 3.11+, Docker Desktop, VS Code, Chrome Browser	API Keys / User Accounts





High level tech stack

Item	Recommendation
Browser	Google Chrome
Notebook Platform	Google Colab
API Keys	Gemini + OpenRouter
GitHub Account	Required
Hugging Face Account	Recommended
Setup Time	~ 15 minutes maximum





The Near Future

Gartner forecasts that by 2028
33% of enterprise software applications will utilize agentic AI
“enabling 15% of day-to-day work decisions to be made
autonomously”.



Learning Objectives:

Module 1

- Understand modern agentic AI architecture
- Learn planning and orchestration patterns
- Build structured enterprise agents
- Implement memory and tool usage
- Design sequential agent workflows
- Compare orchestration patterns



What is an Agentic AI System

There are many ways to describe an agent, but typically, an agent or agentic AI system will exhibit the following elements





What is an Agentic AI System

Planning & Reasoning: Agents can reason and decide about the steps necessary to achieve their objectives. This includes formulating, tracking, and updating their action plans to handle complex tasks (the Reason + Act, ReAct pattern).





What is an Agentic AI System

Modern Agents use LLMs as their reasoning engines, with agents using the LLM to decide the control flow of the application. This is a fundamental aspect of agentic autonomy.





What is an Agentic AI System

Advances in LLMs have allowed for sophisticated reasoning and planning strategies such as





Reasoning and planning strategies for Agentic AI Systems

Reflection

where the agent evaluates past actions and their results to determine future plans or behaviors.

Self-Critic, is a key component of reflection, where the agent critiques its own reasoning or output to identify and correct errors.





Reasoning and planning strategies for Agentic AI Systems

Chain of Thought

is a step-by-step reasoning process in which the agent breaks down complex problems into sequential, logical steps. This can involve multi-step workflows, including ones without human interaction (<https://arxiv.org/abs/2201.11903>).





Reasoning and planning strategies for Agentic AI Systems

Subgoal Decomposition

which involves dividing a main goal into smaller, manageable tasks or milestones to achieve the overall objective .





Agentic AI System Elements

Memory / Statefulness to retain and recall information.

This is either information from previous runs or the previous steps it took in the current run (the reasoning behind their actions, tools they called, the information they retrieved, etc.).

Memory can be session-based short-term or persistent long-term memory.



▶ Agentic AI System Elements

Action and Tool Use

Agents can take action to accomplish tasks and invoke tools as part of the actions.

These can be built-in tools and functions such as browsing the web, conducting complex mathematical calculations, and generating or running executable code in response to a user's query.



▶ Agentic AI System Elements

Action and Tool Use

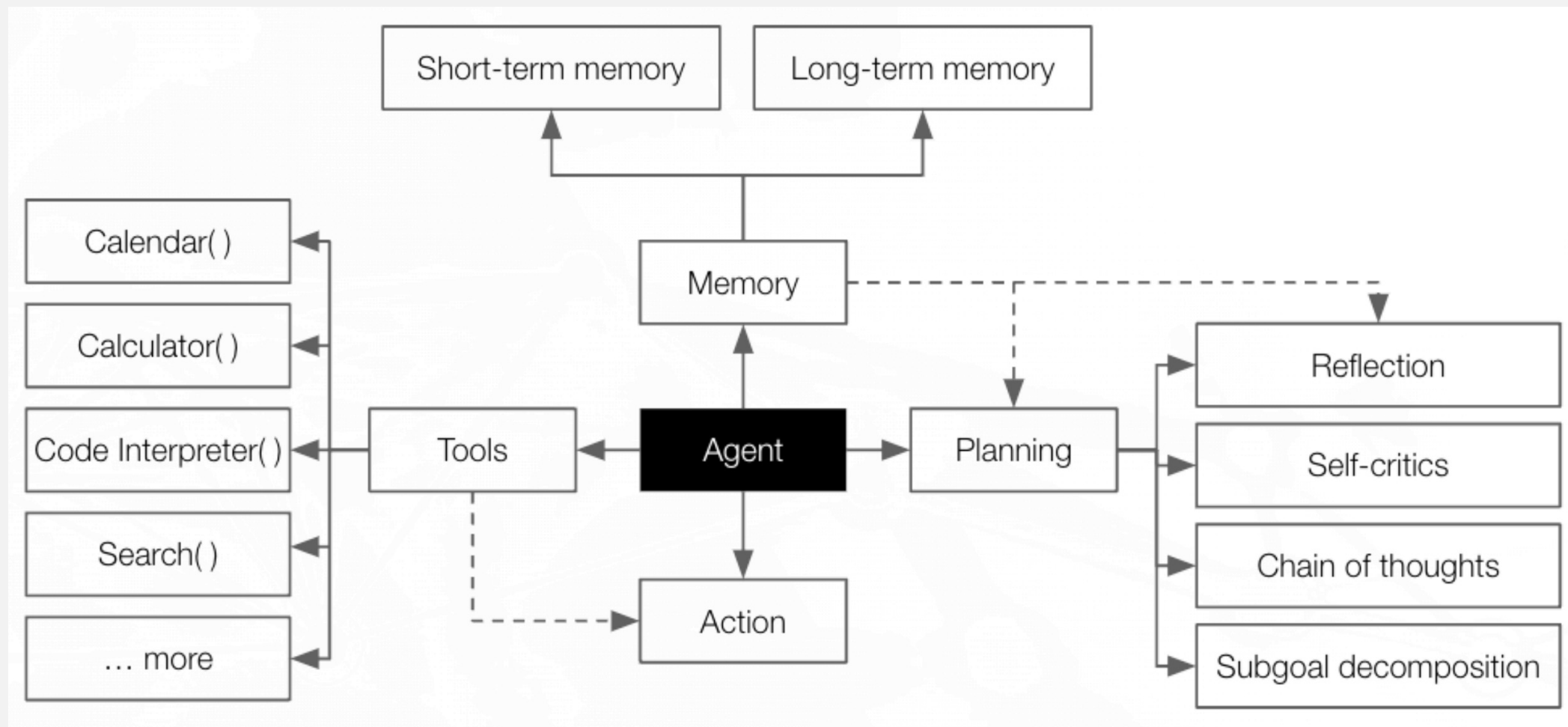
Agents can access more advanced tools via external API calls and a dedicated Tools interface.

The Model Context Protocol (MCP) defines a standardized interface to connect an Agent (MCP Client) with a tool (MCP Server).

These are complemented by augmented LLMs, which offer the tool invocation from code generated by the model via **function calling**, a specialized form of tool use.



Agentic AI System Elements



Agents and LLM Applications

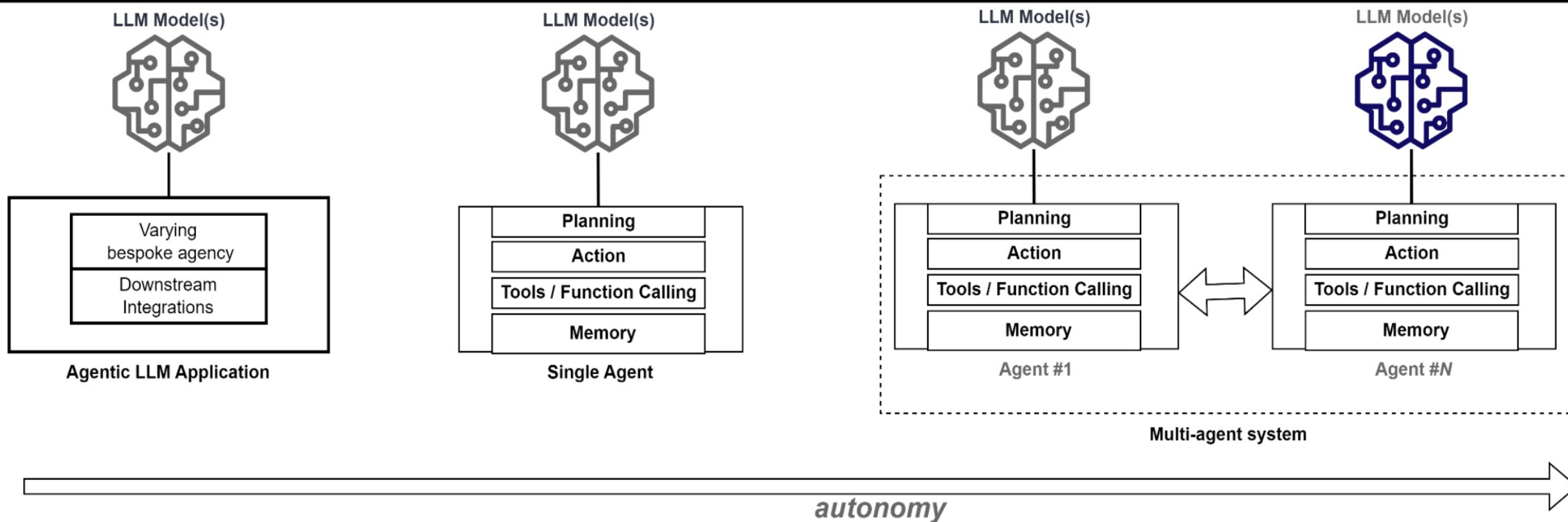
LLM applications can exhibit agency and agentic behavior.

agents can be written as LLM applications with the ability to reason and take action using tools like APIs, databases and so on beyond just generating text-based output.



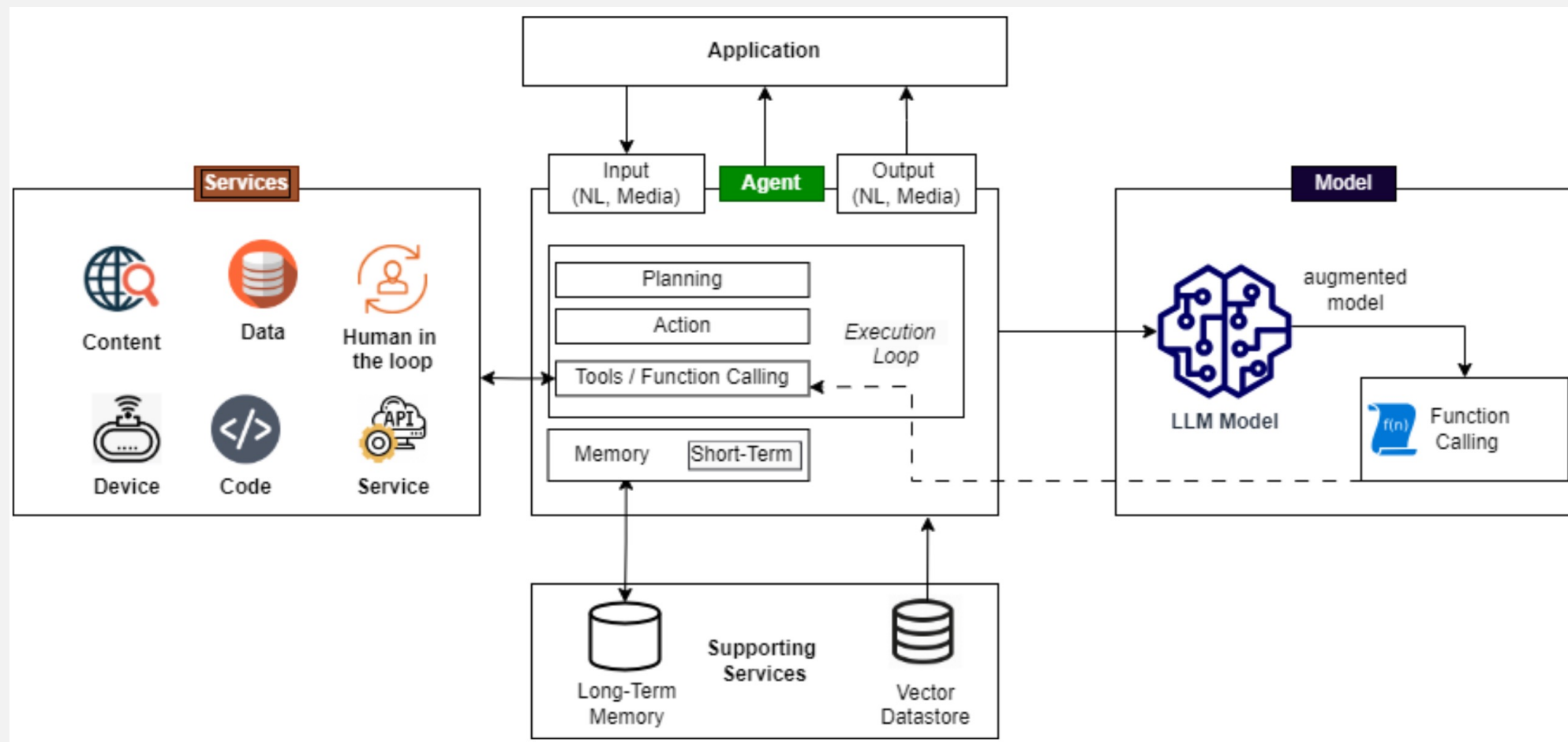
Agents and LLM Applications

Increasingly, developers use agentic AI frameworks, which encapsulate agentic capabilities and offer greater productivity and reuse. Popular frameworks include **LangChain/LangFlow**, **AutoGen**, **CrewAI**, and so on.



Agentic AI Architecture

The following diagram illustrates **single-agent architecture**, highlighting the key deployable components





Agentic AI Architecture

Deployable components will include

1 - An application that has embedded agentic functionality to perform tasks for the user on behalf of the user, often outside a specific user session.



Agentic AI Architecture

Deployable components will include

2 - An agent generally accepts natural language input similar to inputs used for NLP models. This will be textual prompts and optional media such as files, images, sound, or video. The application's code implements the core capabilities and most likely relies on abstractions offered by an agentic framework (LangChain/LangFlow, AutoGen, Crew.AI, and so on).



Agentic AI Architecture

Deployable components will include

3 - One or more LLM models (local or remote) are used for reasoning.



Agentic AI Architecture

4 - Services, including built-in functions, local tools, and local application code, local or remote and external services, will be called in two possible manners:

- a. Function calling and optional Tools interface at the framework/application level
- b. Function calling by an LLM model returning invocation code to the agent.



Agentic AI Architecture

5 - Supporting services, part of the agent infrastructure and core functionality:

- a. External Storage for persistent Long-term memory.
- b. Other data sources include a vector database, other data object storage repositories, and

content used in Retrieval Augmented Generation (RAG). RAG related sources can also be seen as part of the tools, but we highlight it here as a core supporting service that can be used in any LLM application.



Multi-agent Architecture

A **multi-agent architecture** comprises **multiple agents** that can scale or combine **specialist roles** and functionality in an agentic solution.

In both cases, the architecture is similar except for introducing inter- agent communication and, optionally, a coordinating agent.

There is a protocol called Agent2Agent (A2A) that defines a standard specification for multi agent system communication.



Multi-agent Architecture

A **multi-agent architecture** comprises **multiple agents** that can scale or combine **specialist roles** and functionality in an agentic solution.

In both cases, the architecture is similar except for introducing inter- agent communication and, optionally, a coordinating agent.

There is a protocol called Agent2Agent (A2A) that defines a standard specification for multi agent system communication.

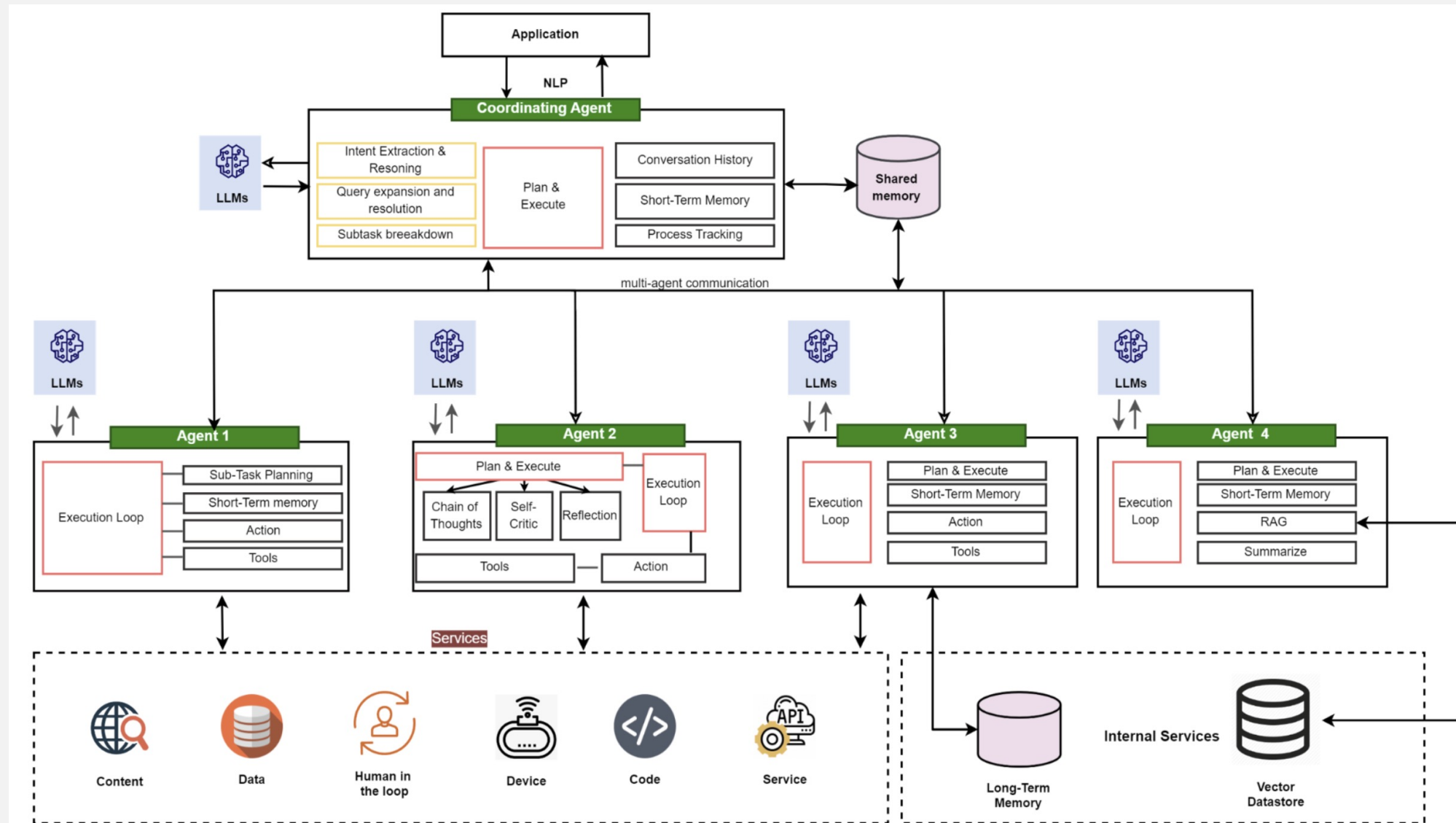


Multi-agent Architecture

The following diagram depicts an example of multi-agent architecture of specialized agent functionality. Specialized functionality is a form of agentic patterns and could be exhibited by any agent depending on the use case.



Multi-agent Architecture



Agentic AI Patterns

Specialized roles and planning strategies contribute to agentic patterns. These patterns are emerging as building blocks that can be combined in a single agent

Pattern	Description
Reflective Agent	Agents that iteratively evaluate and critique their own outputs to enhance performance. Example: AI code generators that review and debug their own outputs, like Codex with self- evaluation.
Task-Oriented Agent	Agents designed to handle specific tasks with clear objectives. Example: Automated customer service agents for appointment scheduling or returns processing.
Hierarchical Agent	Agents are organized in a hierarchy, managing multi-step workflows or distributed control systems. Example: AI systems for project management where higher-level agents oversee task delegation.
Coordinating Agent	Agents facilitate collaboration and coordination and tracking, ensuring efficient execution. Example: a coordinating agent assigns subtasks to specialized agents, such as in AI- powered DevOps workflows where one agent plans deployments, another monitors performance, and a third handles rollbacks based on system feedback
Distributed Agent Ecosystem	Agents interact within a decentralized ecosystem, often in applications like IoT or marketplaces. Example: Autonomous IoT agents managing smart home devices or a marketplace with buyer and seller agents.





Agentic AI Patterns

Pattern	Description
Human-in-the-Loop Collaboration	Agents operate semi-autonomously with human oversight. Example: AI-assisted medical diagnosis tools that provide recommendations but allow doctors to make final decisions
Self-Learning and Adaptive Agents	Agents adapt through continuous learning from interactions and feedback. Example: Co- pilots, which adapt to user interactions over time, learning from feedback and adjusting responses to better align with user preferences and evolving needs.
RAG-Based Agent	This pattern involves the use of Retrieval Augmented Generation (RAG), where AI agents utilize external knowledge sources dynamically to enhance their decision-making and responses. Example: Agents performing real-time web browsing for research assistance.
Planning Agent	Agents autonomously devise and execute multi-step plans to achieve complex objectives. Example: Task management systems organizing and prioritizing tasks based on user goals.
Context- Aware Agent	Agents dynamically adjust their behavior and decision-making based on the context in which they operate. Example: Smart home systems adjusting settings based on user preferences and environmental conditions.



▶ Advanced Design Pattern 1: ReAct (Reason + Act)

- ReAct = interleaving Reasoning traces with Tool Actions
- Loop: Thought → Action → Observation → Thought → Action...
- Enables: grounded reasoning, transparent decision chains, debugging
- Code snippet: LangChain ReAct agent with 3 tools
- Real example: ReAct agent that researches a company (Search→Scrape→Summarize→Report)



Advanced Design Pattern 2: Plan-and-Execute

- Separate planning phase from execution phase
- Planner LLM: generates a step-by-step plan
- Executor Agent: runs each step, reports back
- Re-planner: adjusts plan if execution deviates
- Best for: long-horizon tasks, complex multi-step workflows

Example: Enterprise data pipeline agent
plan 12 steps, execute with error handling



▶ Advanced Design Pattern 3: Reflexion & Self-Critique

- Agent evaluates its own output and iterates
- Components: Actor (generates response), Evaluator (scores it), Reflector (generates feedback)
- Enables: quality improvement without human feedback
- Number of reflection loops: balance quality vs. cost (typically 2-3)
- Code: LangGraph Reflexion loop implementation

Use case: Code generation agents that test and fix their own code



▶ Advanced Design Pattern 4: Hierarchical Delegation

- Manager Agent → Task Assignment → Specialist Sub-Agents
- Manager: decomposes goal, tracks progress, aggregates results
- Specialists: SearchAgent, CodeAgent, DataAgent, ReportAgent
- Communication: structured messages with typed outputs
- Anti-pattern: avoid 'telephone game' — specialist should report directly, not cascade
- Example: Legal document review system (Manager + GDPR-Agent + Compliance-Agent)



▶ The Model Context Protocol (MCP): USB-C for AI

Problem before MCP: custom integration for every tool (3-5x dev time overhead)

- MCP: open standard by Anthropic (Nov 2024) → donated to Linux Foundation (Dec 2025)
- Adopted by: OpenAI, Google, Microsoft, AWS — industry standard by Q1 2026
- Architecture: MCP Client (in agent) ↔ MCP Server (wraps external system)
- Three primitives: Tools (actions), Resources (read-only data), Prompts (templates)
- 5,800+ community MCP servers, 97M monthly SDK downloads (March 2026)





Agent Memory Architecture

4 types of agent memory:

1. In-Context Memory: current conversation window (limited, expensive)
 2. External Memory: vector database (semantic search, unlimited scale)
 3. Episodic Memory: conversation history per user/session
 4. Procedural Memory: tool definitions, workflow instructions
- Memory strategies: summarization, sliding window, entity extraction

Tools: ChromaDB, Pinecone, Weaviate for vector memory



▶ LangGraph: Why State Machines Beat Linear Chains

LangChain (2022): linear chains — $A \rightarrow B \rightarrow C \rightarrow \text{output}$

- Problem: real tasks need branching, looping, error recovery
- LangGraph (2024): directed graph with state — nodes, edges, conditional routing
- Key insight: 'What is the state machine governing this agent's behavior?'
- Used in production: Klarna, Replit, Elastic, and 50,000+ enterprise deployments

Code: minimal StateGraph example (15 lines)



Flow Engineering vs. Prompt Engineering

In 2026, we have moved past the "Prompt Engineering" hype.
For advanced architects, the shift is from writing
the perfect sentence (**Prompt Engineering**)
to building the perfect system (**Flow Engineering**).





Flow Engineering vs. Prompt Engineering

In 2026, we have moved past the "Prompt Engineering" hype.
For advanced architects, the shift is from writing
the perfect sentence (**Prompt Engineering**)
to building the perfect system (**Flow Engineering**).





Flow Engineering vs. Prompt Engineering

Think of Prompt Engineering as "System 1" (intuitive/fast) and Flow Engineering as "System 2" (deliberate/slow) reasoning for AI.





Flow Engineering vs. Prompt Engineering

Think of Prompt Engineering as "System 1" (intuitive/fast) and Flow Engineering as "System 2" (deliberate/slow) reasoning for AI.



Flow Engineering vs. Prompt Engineering

Prompt Engineering: The art of optimizing the input to a single LLM call. It focuses on wording, structure, few-shot examples, and personas. It treats the LLM as a "black box" that you are trying to coax a specific result out of.

Flow Engineering: The discipline of designing multi-step, iterative workflows where the LLM is just one component. It uses loops, conditional logic, and external validation to build the final answer incrementally.



▶ Flow Engineering vs. Prompt Engineering

Feature	Prompt Engineering	Flow Engineering (The 2026 Standard)
Primary Goal	One-shot accuracy.	Process reliability & error recovery.
Philosophy	"Try to say it better."	"Build a system that checks itself."
Complexity	Low (Single turn).	High (State machines, DAGs).
Human Analogy	Giving a chef a single order.	Managing a kitchen with a specialized team.
Failure Mode	Hallucination or "Refusal."	The workflow detects the error and retries.
Architecture	Linear (Prompt → Output).	Cyclic (Nodes, Edges, Reflection Loops).
Job Title	Prompt Engineer (Entry-level).	AI Workflow/Agentic Architect (Expert).



Flow Engineering vs. Prompt Engineering

Multi-Agent Orchestration:

Breaking a problem into a hierarchy. You don't have one prompt that says "Build a website." You have a "Flow" where a UI Agent talks to a Backend Agent, and a PM Agent resolves their conflicts.

4. Why this matters for Architects

Advanced engineers prefer Flow Engineering for three business reasons:

Reliability: You can't rely on an LLM being "smart" every time. A "Flow" makes the system reliable even if the model is slightly inconsistent.

Observability: In a single long prompt, if it fails, you don't know why. In a "Flow," you can see exactly which node (step) failed in your traces (LangSmith/Langfuse).

Cost Efficiency: You can use a very cheap, small model (like Gemini Flash) for 90% of the steps in a "Flow," and only use the expensive model (like Gemini Ultra or GPT-4) for the final "Reasoning" or "Verification" step.



Flow Engineering vs. Prompt Engineering

The Reflection Loop:

Step 1: Agent A generates a draft.

Step 2: Agent B (The Critic) reviews it for errors.

Step 3: Agent A rewrites it based on the critique.

Result: Dramatic increase in quality (e.g., coding accuracy often jumps from 20% to 45%+ using this loop).

The Planning Pattern:

The system first creates a "To-Do list" (a plan) before making any tool calls. If the environment changes, the "Flow" updates the plan dynamically.

Tool-Use (ReAct):

The flow interleaves Thought (Reasoning) and Action (Tool use). The LLM observes the tool's output and updates its state before its next "Thought".

